

## תרגיל 2 ענן

|                                                                                                                                                                   |                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <a href="https://colab.research.google.com/drive/1qeaQIEb42WrSDm5tS0gjRM2VpgghXtU7">https://colab.research.google.com/drive/1qeaQIEb42WrSDm5tS0gjRM2VpgghXtU7</a> | קישור<br>למחברת<br>ההגשה: |
| <a href="https://github.com/Maores/Cloud-Computing-Course">https://github.com/Maores/Cloud-Computing-Course</a>                                                   | קישור<br>לתיקית גיט:      |

| שם חבר הצוות<br>ותפקיד בתרגיל זה                    | משימות שהוקצו                                                                                                                                | משימות שהושלמו                                                  |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| נתן גלבוע - Product Manager &<br>Frontend developer | הגדרת דרישות וארכיטקטורה, בניית<br>ממשק Gradio הכולל אינטגרציה של<br>הפ'יצר הפונקציונלי ואיחוד כל<br>הרכיבים(כולל האינדקס) לאפליקציה<br>אחת. | כל המשימות הושלמו, כולל הגשת<br>הקוד והדו"ח הסופי.              |
| ליאור אסתרקין(מילואים)                              |                                                                                                                                              |                                                                 |
| מאור סבן - Backend developer                        | מימוש מנוע האינדקס, חיבור ל Gemini<br>API , ופיתוח הלוגיקה לשליפת נתונים<br>מה- IoT Server .                                                 | מנוע הRAG עובד, האינדקס נבנה<br>בהצלחה והתקשורת מול הAPI תקינה. |
| לירון זינו – UI & QA                                | עיצוב מערכת, יצירת ערכות נושא<br>ומימושם.<br>בדיקות תקינות לגרפים, בדיקת חריגות<br>ובדיקת תהליך הExport.                                     | העיצוב הוטמע וכל הפונקציות נבדקו<br>ונמצאו כתקינות.             |

**חלק ראשון : בניית מנוע חיפוש**

## תרגיל 2 ענן

### **בניית אינדקס - משימות**

1. האינדקס מומש בcollab.
2. עבור רשימת ה- Stopwords בחרנו להשתמש ברשימה המובנית של ספריית NLTK הכוללת 198 מילים נפוצות באנגלית. הבחירה ברשימה סטנדרטית זו נועדה לסנן מילים בעלות תדירות גבוהה אך ערך סמנטי נמוך (כגון מילות יחס וחיבור), כך שסינון זה מקטין את נפח האינדקס וממקד את החיפוש במונחים בעלי משמעות, ובכך משפר את יעילות ודיוק המערכת.
- המילים שרשימת ה- Stopwords כוללת הינם:  
a, about, above, after, again, against, ain, all, am, an, and, any, are, aren, aren't, as, at, be, because, been, before, being, below, between, both, but, by, can, couldn, couldn't, d, did, didn, didn't, do, does, doesn, doesn't, doing, don, don't, down, during, each, few, for, from, further, had, hadn, hadn't, has, hasn, hasn't, have, haven, haven't, having, he, he'd, he'll, he's, her, here, hers, herself, him, himself, his, how, i, i'd, i'll, i'm, i've, if, in, into, is, isn, isn't, it, it'd, it'll, it's, its, itself, just, ll, m, ma, me, mightn, mightn't, more, most, mustn, mustn't, my, myself, needn, needn't, no, nor, not, now, o, of, off, on, once, only, or, other, our, ours, ourselves, out, over, own, re, s, same, shan, shan't, she, she'd, she'll, she's, should, should've, shouldn, shouldn't, so, some, such, t, than, that, that'll, the, their, theirs, them, themselves, then, there, these, they, they'd, they'll, they're, they've, this, those, through, to, too, under, until, up, ve, very, was, wasn, wasn't, we, we'd, we'll, we're, we've, were, weren, weren't, what, when, where, which, while, who, whom, why, will, with, won, won't, wouldn, wouldn't, y, you, you'd, you'll, you're, you've, your, yours, yourself, yourselves.
3. לצורך בניית האינדקס בחרנו להשתמש ב Stemming, שיושם באמצעות אלגוריתם Porter Stemmer מספריית NLTK. תהליך זה ממיר הטיות שונות של מילים לשורש המשותף שלהם (לדוגמא plant לplant), ובכך מקטין את גודל האינדקס ומשפר את יכולת האחזור על ידי זיהוי מילים בעלות משמעות דומה.
4. מנגנון RAG מומש בcollab.

## תרגיל 2 ענן

**חלק שני : בניית מסכים להצגה בכיתה**

**חלק 2 – משימות**

**1. התייחסות ל-8 כללי הזהב של שניידרמן, כיצד המערכת שלכם מבטאת אותם?**

1. **שמירה על עקביות: (Consistency)**
  - השתמשנו בספריית Gradio שדואגת שכל הכפתורים, הפונטים ותיבות הטקסט ייראו אותו דבר בכל המסכים (דשבורד וצ'אט). המשתמש לא צריך לנחש איפה ללחוץ כי העיצוב אחיד.
2. **שימוש בקיצורי דרך למשתמשים מנוסים: (Universal Usability)**
  - חילקנו את המערכת למסכים ברורים שניתן לבחור כל אחד מהם באופן עצמאי, כך שאפשר לעבור ממסך הניטור מצב צמחים למסך הצ'אט בלי לחפש בתפריטים מסובכים.
3. **משוב אינפורמטיבי: (Informative Feedback)**
  - בכל פעולה שהמשתמש עושה, המערכת מגיבה (למשל בצ'אטבוט רואים את הטקסט נכתב/ מוצג סימן שהמערכת "חושבת", וכשלוחצים על ייצוא ל CSV-מקבלים חיווי שהקובץ מוכן להורדה).
4. **סגירת מעגל: (Closure)**
  - פעולות יש התחלה וסוף ברורים. למשל, בתהליך שאלת שאלה ב RAG – המשתמש שואל שאלה והמערכת עונה. הפעולה הסתיימה והמשתמש מבין שהוא יכול להמשיך הלאה.
5. **מניעת שגיאות וטיפול פשוט: (Error Prevention/Handling)**
  - במקום לתת למשתמש להקליד פקודות מסובכות שמנו כפתורים מוכנים (כמו refresh או export כדי למנוע טעויות הקלדה. אם יש בעיה בחיבור, המערכת לא קורסת אלא פשוט לא מציגה נתון (או מציגה הודעה ידידותית).
6. **אפשרות חרטה/ביטול פעולה: (Permit easy reversal of actions)**
  - במסך הצ'אט יש אפשרות לנקות את ההיסטוריה ("Clear") ולהתחיל מחדש אם השיחה לא הלכה לכיוון הרצוי, או לרענן את הנתונים אם המידע לא נראה עדכני.
7. **תחושת שליטה למשתמש: (Internal Locus of Control)**
  - המשתמש הוא זה שיוזם את הפעולות, מאחר והמערכת לא עושה דברים "מאחורי הגב" (כמו רענון אוטומטי פתאומי שמזיז את המסך), אלא מחכה שהמשתמש ילחץ על כפתור הרענון או ישלח שאלה.
8. **הפחתת העומס על הזיכרון: (Reduce short-term memory load)**
  - כל המידע מוצג מול העיניים בדשבורד כך שהמשתמש לא צריך לזכור מה הייתה הטמפרטורה לפני דקה כדי להשוות כי הכל מוצג בטבלה/גרף מולו, והכותרות של כל מדד ברורות (למשל: "לחות", "טמפרטורה").

## תרגיל 2 ענן

### 2.טבלת המשובים

| הערות משוב                                       | האם התבצע שינוי באפליקציה בעקבות ההערה? | נימוק                                                                                                                                                                                                        |
|--------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| הוספת גרפים ויזואליים לדשבורד                    | לא בוצע כרגע (אולי יבוצע בעתיד)         | הוספת ויזואליזציה גרפית דורשת אינטגרציה עם ספריות ועיבוד נתונים נוסף. עקב מגבלת הזמן בהגשה הנוכחית ואי-זמינות אנשי הצוות של הפרויקט(מילואים), תיעדפנו את הפונקציונליות של המודל.                             |
| שינוי אופן טעינת ההיסטוריה                       | לא בוצע                                 | מנגנון ההיסטוריה הנוכחי מבוסס על ארכיטקטורת ה-DB-שבנינו בשלב האפיון. שינוי אופן הטעינה כעת דורש שינוי בסכמת בסיס הנתונים, דבר שעלול לגרום באגים ולערער את יציבות המערכת בשלב קריטי זה.                       |
| שינוי מבנה קובץ ה-CSV המיוצא                     | לא בוצע                                 | פורמט CSV-הנוכחי תוכנן ספציפית כדי להתאים לדרישות שהוגדרו מראש ולמערכות חיצוניות שאמורות לקרוא אותו. שינוי המבנה עלול לפגוע בתאימות הנתונים                                                                  |
| אפשרות לשינוי עיצוב                              | לא בוצע כרגע (יבוצע בעתיד)              | מפאת חוסר הזמן לאחר קבלת המשובים, החלטנו להוסיף את המנגנון לאפשרות שינוי העיצוב בהמשך העבודה על הפרויקט. זהו מנגנון איכותי שיכול להנעים את חווית המשתמש במערכת ולכן קיבלנו החלטה לבצע אינטגרציה עבורו בעתיד. |
| הוספת רענון אוטומטי (Auto-Refresh) לנתוני החיפוש | לא בוצע כרגע (יבוצע בעתיד)              | בגרסה הבאה נוסיף למערכת עדכון בזמן אמת של נתוני החיפוש.                                                                                                                                                      |

### 3.ציון SUS, מה מעיד הציון?

ציון ה-SUS: 89(חושב הצורה הבאה):

- אופן החישוב:** לכל נבדק בוצע נרמול של 10 שאלות ה-SUS. בשאלות החיוביות (אי-זוגיות) הופחת 1 מהתשובה, בשליליות (זוגיות) הופחתה התשובה מ-5, וסכום התוצאות הוכפל ב-2.5.
- התפלגות הציונים:** מניתוח הנתונים עולה כי הרוב המכריע של המשיבים (כ-8 משתמשים) העניקו למערכת ציון "מושלם" (100), בעוד מספר מצומצם (כ-3 משתמשים) העניקו ציונים בטווח הבינוני (50-70).
- התוצאה הסופית:** שקלול הציונים הללו הניב ציון ממוצע של 89.2, המוגדר כציון "מצוין" וגבוה משמעותית מהממוצע.

**משמעות הציון:** ציון זה משקף רמת שמישות מצוינת ומעיד כי המשתמשים תופסים את המערכת כאינטואיטיבית, יציבה וקלה ללמידה ללא צורך בתמיכה טכנית חיצונית.

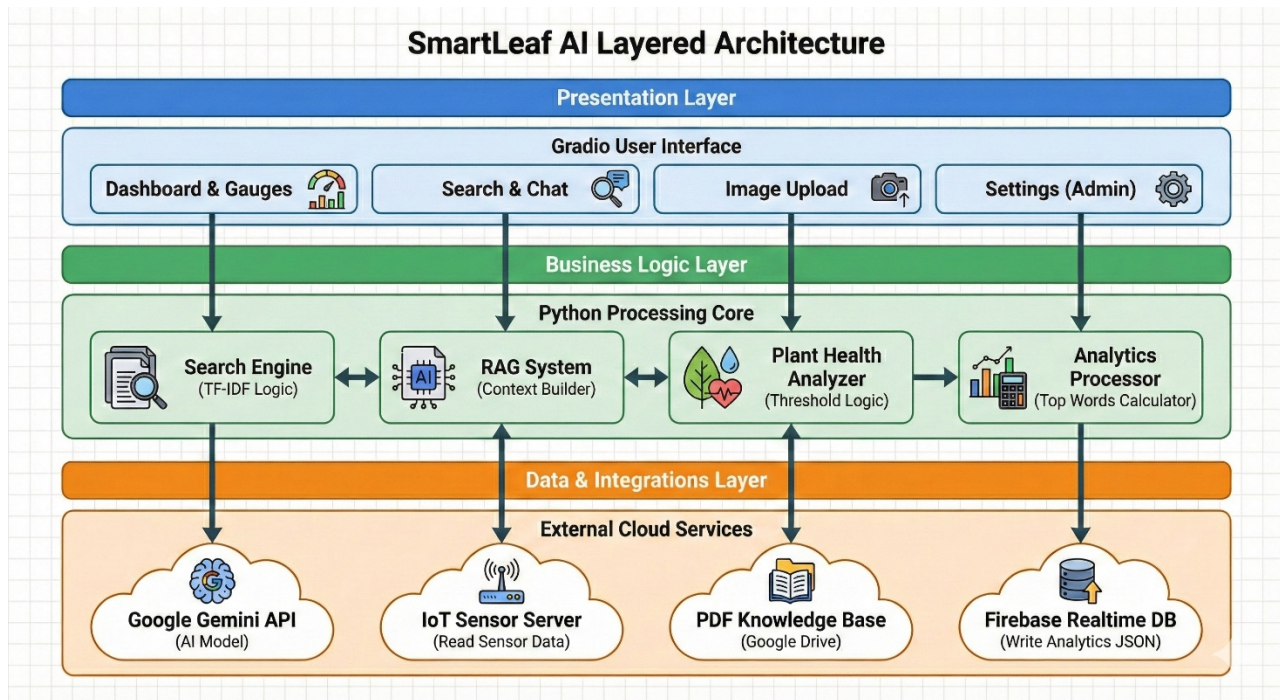
## תרגיל 2 ענן

### 4. הגדרת 3 מדדים להצלחת המערכת

1. יכולת גדילה (Scalability): המערכת צריכה לדעת להתמודד עם הוספה של עוד ועוד חיישנים וDATA ללא האטת פעילות המערכת, כדי שנוכל להרחיב את השימוש בעתיד בלי לשנות את התשתית.
2. אמינות וזמינות (Reliability): הדגש הוא שהמערכת תעבוד בצורה רציפה וששום מידע שמגיע מהחיישנים לא ילך לאיבוד בדרך, כדי שהמשתמש יוכל לסמוך על הנתונים.
3. זמן תגובה מהיר (Performance): חשוב לנו שהמשתמש יראה את הנתונים על המסך כמעט באותו רגע שהם נמדדו (עד 2 שניות), כדי לאפשר ניטור אמיתי בזמן אמת.

## תרגיל 2 ענן

### 1. דיאגרמת ארכיטקטורה



#### • סוג הארכיטקטורה

הארכיטקטורה שנבחרה עבור המערכת היא **ארכיטקטורת שכבות (Layered Architecture)**. בחרנו בארכיטקטורה זו מכיוון שהיא מאוד נוחה ובעלת מגוון יתרונות, כאשר אחד מיתרונותיה הבולטים הוא השימוש במבנה היררכי ומודולרי - כך שישנה הקלה משמעותית בתחזוקת המערכת, ביצוע בדיקות והרחבת הפונקציונליות בעתיד, והיכולת לבצע שינויים בשכבה מסוימת ללא השפעה או צורך בשינוי קוד עמוק בשכבות הלוגיקה והנתונים.

#### • פירוט חלקי הקוד המתייחסים לכל חלק בארכיטקטורה

- **שכבת התצוגה (Presentation Layer)** – שכבה זו אחראית על האינטראקציה מול המשתמש, קבלת החלטות והצגת הפלט הוויזואלי.

- **מיקום בקוד:** (User Interface Construction) **Cell 8**.
- **רכיבים ופונקציות:**

- שימוש בספריית **gradio** לבניית הממשק (**gr.Blocks**, **gr.Tabs**).
- הגדרת רכיבי **UI**: כפתורים, גרפים (**gr.Plot**), העלאת תמונות, ותיבות טקסט.
- **עיצוב (Styling)**: משתנה **APPLICATION\_CSS** וסקריפט **THEME\_SWITCHER\_JS**.
- לשינוי ערכות נושא.
- **פונקציות עזר לתצוגה בלבד**: **generate\_gauge\_placeholder** (יצירת HTML ויזואלי).

- **שכבת הלוגיקה העסקית (Business Logic Layer)** - בשכבה זו מתבצעים העיבודים, החישובים, וקבלת ההחלטות לפני שהמידע נשמר או מוצג.

## תרגיל 2 ענן

- מיקום בקוד : Cell 3, Cell 4, Cell 5, Cell 7.
- רכיבים ופונקציות:
  - מנוע חיפוש (Search Engine): פונקציות חישוב `calculate_tfidf_score` ו-`build_complete_search_index` (בניית אינדקס הפוך ודירוג מסמכים).
  - מודל RAG: הפונקציה `generate_rag_answer` שמבצעת את בניית ה-Context והנדסת ה-Prompt עבור ה-AI.
  - ניתוח בריאות הצמח: הפונקציה `analyze_plant_health_status` שמכילה לוגיקה של חוקים (Rules) לבדיקת ספים של טמפרטורה ולחות.
  - עיבוד נתונים לאנליטיקה: הפונקציה `upload_top_words_to_firebase` שמבצעת עיבוד סטטיסטי למציאת 5 המילים הנפוצות ביותר בכל מאמר.
- שכבת הנתונים והאינטגרציות (Data & Integration Layer) – שכבה זו אחראית על התקשורת עם הממשקים החיצונים, שמירת הנתונים ושליפתם.
  - מיקום בקוד : Cell 2, Cell 6, Cell 7.
  - רכיבים ופונקציות :
    - Google Gemini API: הגדרת המודל `genai.GenerativeModel` ושליחת בקשות ל-LLM.
    - IoT Server: הפונקציה `fetch_iot_sensor_data` שמבצעת קריאות HTTP GET לשרת החיישנים החיצוני.
    - Firebase Realtime DB: ביצוע קריאת `requests.put` לשמירת ה-JSON של המילים הנפוצות בענן.
    - PDF Knowledge Base: טעינת קבצים גולמיים מ-Google Drive באמצעות `download_pdf_from_url`.

## תרגיל 2 ענן

### חלק שלישי: פיצ'ר לבחירתכם

- **פיצ'ר פונקציונלי שנבחר** - ייצוא נתוני דגימות החיישנים אל תוך קובץ CSV ואפשרות להורדתו.
- **תיאור הפיצ'ר**
  - הוספנו למערכת יכולת ייצוא נתונים שמאפשרת להוריד את היסטוריית הקריאות של החיישנים (אדמה/לחות/טמפרטורה) לקובץ בפורמט CSV. זהו פיצ'ר פונקציונלי שמאפשר מחקר וניתוח נתונים חיצוני, מה שאינו מתאפשר ישירות מה Dashboard עצמו. המערכת מושכת את המידע הרלוונטי מהשרת ומייצרת קובץ מוכן להורדה.
- **מיקום המימוש בקוד** – הלוגיקה של הפיצ'ר ממושת בשני חלקים עיקריים במחברת (Cell 7, Cell 8):

```
# FIXED: CSV Export Function - Removed Unit column
def export_to_csv(feed_name, limit):
    """
    Exports IoT data to a CSV file (without Unit column)
    """
    df = fetch_feed_data(feed_name, limit=int(limit))

    if df.empty:
        return None, "⚠️ No data available to export"

    # Add feed_type column to clearly identify the data source
    df['feed_type'] = feed_name.capitalize()

    # Reorder columns - REMOVED unit column
    df = df[['feed_type', 'created_at', 'value']]

    # Rename columns for clarity
    df.columns = ['Feed Type', 'Timestamp', 'Value']

    # Create a temporary CSV file
    timestamp = time.strftime("%Y%m%d_%H%M%S")
    filename = f"smartleaf_{feed_name}_{timestamp}.csv"

    # Save to CSV
    df.to_csv(filename, index=False)

    # Create success message with summary
    avg_val = df['Value'].mean()
    min_val = df['Value'].min()
    max_val = df['Value'].max()
    message = f"✅ **Export Successful**

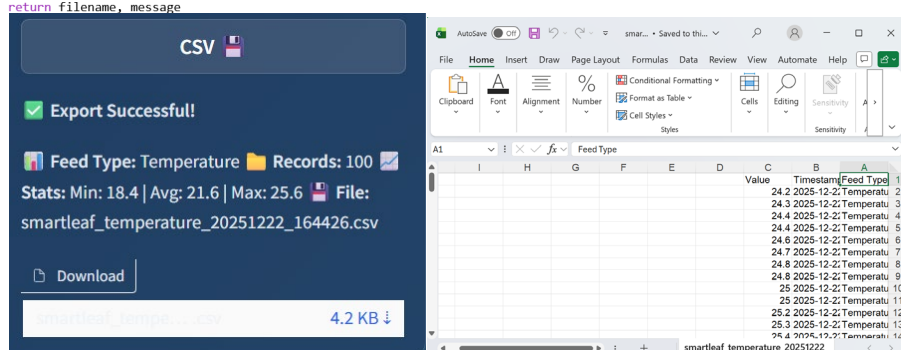
    **Feed Type:** {feed_name.capitalize()}
    **Records:** {len(df)}
    **Stats:** Min: {min_val:.1f} | Avg: {avg_val:.1f} | Max: {max_val:.1f}
    **File:** {filename}"""

    return filename, message
```

```
# --- TAB 2: IOT ANALYTICS ---
with gr.Tab("IoT Analytics 📊"):
    with gr.Row():
        with gr.Column(scale=1, elem_classes="sensor-card"):
            gr.Markdown("### ⚙️ Controls")
            iot_dd = gr.Dropdown(["humidity", "soil", "temperature"], label="Feed", value="humidity")
            iot_limit = gr.Slider(minimum=1, maximum=100, value=10, step=1, label="Samples")
            with gr.Row():
                iot_btn = gr.Button("Fetch 🔄", variant="primary", scale=2)
                export_btn = gr.Button("CSV 📄", variant="secondary", scale=2)
            export_status = gr.Markdown("", visible=True)
            csv_file = gr.File(label="Download", visible=False)

        with gr.Column(scale=3, elem_classes="sensor-card"):
            gr.Markdown("### 📊 Visualization")
            iot_plot = gr.Plot(label="Live Data")
```

- **ביטוי הפיצ'ר בחלק המוצג למנהל** – במסך IoT Analytics, לצד כפתור טעינת הנתונים הגרפיים – נוסף כפתור ייעודי בשם **Export CSV**. בעת לחיצה על הכפתור נוצר קובץ CSV, מופיע רכיב הורדת הקובץ המיועד לשמירה על המחשב האישי ביחד עם סיכום סטטיסטי קצר של הנתונים שיוצאו.



- בנוסף לפיצ'ר, הוספנו אפשרות לבחור Theme בתוך מסך Settings, וגם כפתור המאפשר להעלות את האפליקציה ל Firebase
- מומלץ לפתוח את המחברת דרך הלינק כמו בתמונה מטה:

